

📖 CYBERSECURITY NOTES

Beginner-Friendly Study Notes • Linux • Python • Password Security • Revision

What this book covers	Why it matters
Linux files, directories & terminal	Foundation for security work
Permissions, users & groups	Controls who can access data
Python security basics	Helps write safer programs
Passwords, strength & hashing	Protects accounts and credentials
Revision + practice	Builds confidence through repetition

Study tip: Read the Simple English first, then the Tinglish explanation, and finally type the examples yourself in a safe practice environment.

1. Linux: Files & Directories

Simple English: Linux stores information in files and organizes those files inside directories (folders). The filesystem starts at the root directory, written as /. A path tells Linux where an item is located. An absolute path starts from /, while a relative path starts from the current directory.

Tinglish: Linux lo files and folders ni organize chestundi. / anedi root directory. Current location ni pwd tho telusukovachu; folders madhya move avvadaniki cd use chestham; files/folders choodadaniki ls use chestham.

Easy real-life examples

- Your college project folder can contain subfolders such as notes, code and output.
- A photo saved as /home/thara/Pictures/photo.jpg has a clear path to the file.
- Like a cupboard: directory = shelf/folder, file = item stored inside it.

Important Linux commands

Command	Use
pwd	Print the current working directory
ls	List files and directories
ls -la	Show detailed listing, including hidden files
cd folder	Move into a directory
cd ..	Move to the parent directory
cd ~	Go to the home directory
mkdir name	Create a directory
touch file.txt	Create an empty file
cp source dest	Copy a file or directory
mv old new	Move or rename
rm file	Remove a file
rmdir folder	Remove an empty directory

Small terminal / code example

```
$ pwd
/home/thara
$ mkdir cyber_notes
$ cd cyber_notes
$ touch passwords.txt
$ ls
passwords.txt
```

Key points to remember

- The root directory is /, not a normal user folder.
- pwd tells you where you are.
- ls shows what is inside the current directory.
- cd changes your location.
- Use rm carefully because deleted files may not be recoverable.
- Hidden Linux files commonly begin with a dot, such as .bashrc.

2. Linux Revision + Terminal Commands

Simple English: The terminal is a text-based interface used to communicate with the operating system. Commands usually have a command name followed by options and arguments. Learning a small set of commands makes Linux work faster and more precise.

Tinglish: Terminal ante Linux tho text commands use chesi work chese place. Command + option + argument structure common ga untundi. First safe commands practice cheyadam important.

Easy real-life examples

- Instead of opening a graphical file manager, you can use `ls` and `cd` to navigate quickly.
- `grep` can search for a word inside text, which is useful when checking logs.
- `history` can help you review commands you previously typed.

Important Linux commands

Command	Use
<code>whoami</code>	Show the current username
<code>id</code>	Show user and group IDs
<code>clear</code>	Clear terminal screen
<code>cat file</code>	Display file contents
<code>less file</code>	Read a file page by page
<code>head file</code>	Show first lines
<code>tail file</code>	Show last lines
<code>grep word file</code>	Search for a word/pattern
<code>find path -name name</code>	Find matching files
<code>history</code>	Show command history
<code>man command</code>	Open the manual for a command
<code>echo text</code>	Print text

Small terminal / code example

```
$ whoami
thara
$ echo "Cybersecurity practice"
Cybersecurity practice
$ cat notes.txt
Linux is useful for security learning.
```

Key points to remember

- Terminal commands are case-sensitive.
- Use `man` command to learn the correct options.
- Arguments tell a command what to act on.
- Do not paste unknown commands into a terminal.
- Practice in a test directory rather than important system folders.

3. Files, Permissions & chmod

Simple English: Linux permissions control who may read, write, or execute a file. The three common permission groups are user/owner, group, and others. The three basic permission types are read (r), write (w), and execute (x). chmod changes permissions.

Tinglish: Tinglish lo cheppali ante, file ni evaru chadavali, modify cheyali, run cheyali ane control permissions. Owner, group, others ane three categories untayi. chmod tho permissions change chestham.

Easy real-life examples

- A private notes file may be readable only by its owner.
- A script that should run needs execute permission.
- A shared project file may give the team group read/write access while limiting others.

Important Linux commands

Command	Use
ls -l	View permissions and ownership
chmod 600 file	Owner read/write; others no access
chmod 644 file	Owner read/write; others read
chmod 700 folder	Owner full access
chmod +x script.sh	Add execute permission
chmod -x script.sh	Remove execute permission

Small terminal / code example

```
$ ls -l notes.txt
-rw-r--r-- 1 thara students 120 Aug 21 10:20 notes.txt

$ chmod 600 notes.txt
$ ls -l notes.txt
-rw----- 1 thara students 120 Aug 21 10:20 notes.txt
```

Key points to remember

- r = read, w = write, x = execute.
- Permission order is owner, group, others.
- 644 is common for normal non-secret files.
- 600 is useful for files that should be private to the owner.
- Do not give execute permission unless it is needed.
- chmod changes permissions; it does not change the file's contents.

Numeric chmod idea

The numbers are based on r=4, w=2, x=1. Therefore 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--, and 0 = ---.

4. Users, Groups & Permissions

Simple English: Linux can have many user accounts. Groups allow administrators to assign access to multiple users efficiently. Ownership and group membership work together with file permissions to control access.

Tinglish: Linux system lo different users untaru. Same project lo work chese users ni group lo pettachu. File owner and group permissions use chesi access control chestham.

Easy real-life examples

- A college lab can have a students group so many student accounts can access shared learning material.
- A project directory can belong to a project group, allowing team members to work on it.
- An administrator account has more control than a normal user, so it should be used carefully.

Important Linux commands

Command	Use
whoami	Current user
id	User ID and group membership
groups	Groups of the current user
sudo command	Run an administrative command when permitted
useradd name	Create a user (administrator task)
passwd name	Set/change a user's password (administrator task)
groupadd name	Create a group (administrator task)
usermod -aG group user	Add a user to a supplementary group
chown user file	Change file owner (administrator task)
chgrp group file	Change file group

Small terminal / code example

```
$ id
uid=1001(thara) gid=1001(students) groups=1001(students)

$ groups
students lab
```

Key points to remember

- Users identify accounts; groups simplify shared access.
- Least privilege means giving only the access that is actually needed.
- sudo should be used carefully because it can perform powerful administrative actions.
- Never share your password just to let someone access a file.
- Always understand ownership and permissions before changing them.

5. Python Security Basics

Simple English: Python is useful for cybersecurity because it can automate repetitive tasks, parse files, validate input, work with hashes, and build security tools. Secure Python programming also means treating external input as untrusted and avoiding dangerous shortcuts.

Tinglish: Python cybersecurity lo automation ki chala useful. Files read cheyadam, input validate cheyadam, hashing, logs process cheyadam lantivi cheyachu. User input ni direct ga trust cheyakudadhu.

Easy real-life examples

- A password checker can evaluate length and character variety.
- A log parser can count failed login events.
- A program can validate that an input is a number before performing a calculation.

Small terminal / code example

```
def safe_integer(text):
    try:
        return int(text)
    except ValueError:
        return None

value = safe_integer("42")
print(value)

# Avoid dangerous patterns such as:
# eval(user_input)
# exec(user_input)
```

Key points to remember

- Validate input before using it.
- Avoid eval() and exec() on untrusted input.
- Do not hard-code real passwords, API keys, or secrets in source code.
- Use libraries that implement security correctly instead of inventing cryptography.
- Keep dependencies updated and install packages from trusted sources.
- Log security events carefully without exposing secrets.

Safe coding mindset

Think: What can the user control? What happens if the input is empty, huge, malformed, or unexpected? Security starts by assuming external input can be wrong or malicious.

6. Safe Password Concepts

Simple English: A strong password is long, unique, and difficult to guess. Passwords should not be reused across important accounts. A password manager can help create and store unique passwords. Multi-factor authentication (MFA) adds another layer of protection.

Tinglish: Strong password ante just symbols ekkuva undadam kaadu. Length and uniqueness important. Oka site password ni vere site lo reuse cheyakudadhu. MFA add chesthe extra security layer vastundi.

Easy real-life examples

- Using the same password for email and social media is risky because one breach can affect both.
- A long unique passphrase is generally easier to remember and can be harder to guess than a short predictable password.
- MFA can require a second factor after the password.

Small terminal / code example

```
Example of a safer pattern:
long + unique + unpredictable

Avoid patterns such as:
Name@123
Password123
12345678
birthday-based passwords
```

Key points to remember

- Use a unique password for every important account.
- Prefer long passwords/passphrases.
- Avoid names, birthdays, phone numbers, and common words used predictably.
- Never share passwords in chats or notes that others can access.

- Turn on MFA when available.
- Do not use a password example from study notes as your real password.

7. Password Strength Checker

Simple English: A password strength checker estimates whether a password meets basic rules such as minimum length and variety of characters. It is only an educational indicator; a real password system should use stronger, context-aware checks and secure password storage.

Tinglish: Tinglish lo, checker password ni length, uppercase, lowercase, digit, special character lantivi check chestundi. Idi basic indicator matrame; real application lo password ni plain text ga store cheyakudadhu.

Easy real-life examples

- A study program can warn when a test password is too short.
- A website may reject passwords that are extremely common or found in known breach lists.
- Strength checking should not encourage users to create predictable symbol substitutions.

Small terminal / code example

```
import string

def password_strength(p):
    score = 0

    if len(p) >= 12:
        score += 1
    if any(c.islower() for c in p):
        score += 1
    if any(c.isupper() for c in p):
        score += 1
    if any(c.isdigit() for c in p):
        score += 1
    if any(c in string.punctuation for c in p):
        score += 1

    if score <= 2:
        return "Weak"
    elif score <= 4:
        return "Moderate"
    return "Strong"

print(password_strength("Amigo!Study2026"))
```

Key points to remember

- Length is an important factor.
- Character variety can improve basic strength checks.
- Common or breached passwords should still be rejected even if they look complex.
- A checker should never print or log users' real passwords.
- Use test passwords for practice.

8. Password Security + Hashing

Simple English: Passwords should not normally be stored as plain text. A secure password system stores a password hash produced by a password-hashing algorithm with a unique salt. During login, the entered password is processed again and compared with the stored verification data.

Tinglish: Password ni plain text ga database lo store cheyadam unsafe. Salt tho password hashing algorithm use chesi verification data store chestham. Login appudu entered password ni same process tho verify chestham.

Easy real-life examples

- A website database leak is less damaging when passwords are stored using an appropriate password-hashing scheme rather than plain text.
- A salt makes identical passwords produce different stored hash values.
- Modern applications should use dedicated password hashing such as Argon2id, scrypt, or bcrypt according to current guidance.

Small terminal / code example

```
# Educational demonstration only:
import hashlib

password = b"example-password"
digest = hashlib.sha256(password).hexdigest()

print(digest)

# For real password storage, do NOT use plain SHA-256 alone.
# Use a dedicated password-hashing library/algorithm
# such as Argon2id, scrypt, or bcrypt.
```

Key points to remember

- Hashing is designed to be one-way for password verification.
- Encryption is different: encryption is designed to be reversible with a key.
- A salt should be unique per password.
- Fast general-purpose hashes are not ideal for password storage.
- Never store or print real passwords in plain text.
- Use established password-hashing libraries rather than writing your own algorithm.

Hashing vs encryption

Hashing: one-way transformation used for verification. Encryption: reversible transformation used when authorized parties need to recover the original data.

9. Cybersecurity Revision

Simple English: Cybersecurity is the practice of protecting systems, networks, applications, devices, and data from unauthorized access, misuse, disruption, or damage. A beginner should understand the CIA triad: confidentiality, integrity, and availability.

Tinglish: Cybersecurity ante systems, data, networks, applications ni unauthorized access and attacks nunchi protect cheyadam. CIA triad lo C=Confidentiality, I=Integrity, A=Availability.

Easy real-life examples

- Confidentiality: only authorized people should see private student records.
- Integrity: marks in a database should not be changed without authorization.
- Availability: a college portal should remain usable when students need it.

Small terminal / code example

```
CIA Triad
C = Confidentiality -> keep information private
I = Integrity       -> keep information correct
A = Availability    -> keep services/data accessible
```

Key points to remember

- Authentication asks: Who are you?

- Authorization asks: What are you allowed to do?
- Least privilege limits unnecessary access.
- Phishing tricks people into revealing information.
- Malware is malicious software; examples include viruses, worms, trojans, and ransomware.
- Backups help recovery from data loss or ransomware.
- Updates and patches fix known vulnerabilities.
- Security is a combination of technology, processes, and user awareness.

Common threats

Phishing uses deceptive messages or pages. Malware is harmful software. Brute-force attacks repeatedly guess credentials. Social engineering manipulates people. Misconfiguration can expose data even without sophisticated malware.

10. Linux Revision + Practice

Simple English: This final revision connects the earlier topics. The goal is to navigate safely, inspect permissions, understand users/groups, and practice small security-oriented tasks without touching systems you do not own or have permission to test.

Tinglish: Last revision lo commands ni combine chesi practice chestham. Own lab/test folder lo practice cheyali. Permission lekunda vere systems ni test cheyakudadhu.

Easy real-life examples

- Create a cyber_lab folder, add sample files, inspect permissions, and change only the permissions you understand.
- Create a small Python script that checks test-password strength without storing the input.
- Use grep to search a sample log file for the word FAILED.

Important Linux commands

Command	Use
mkdir cyber_lab	Create practice directory
cd cyber_lab	Enter practice directory
touch notes.txt script.sh log.txt	Create sample files
ls -l	Inspect permissions
chmod 600 notes.txt	Make notes private
chmod +x script.sh	Make a script executable
grep FAILED log.txt	Search sample log entries
cat log.txt	Display a small test log
pwd	Confirm location

Small terminal / code example

```
$ mkdir cyber_lab
$ cd cyber_lab
$ touch notes.txt script.sh log.txt
$ echo "FAILED login for Amigo" > log.txt
$ grep FAILED log.txt
FAILED login for Amigo
$ chmod 600 notes.txt
$ ls -l notes.txt
-rw----- 1 thara students 0 Aug 21 10:30 notes.txt
```

Key points to remember

- Practice only on systems, files, and accounts you own or are authorized to use.
- Confirm your current path before running destructive commands.
- Use `ls -l` to inspect permissions before changing them.
- Use test data instead of real passwords or private information.
- Understand the command before using `sudo`.
- Cybersecurity learning should improve protection, not enable unauthorized access.

ONE-PAGE QUICK REVISION

Linux navigation: pwd = where am I? • ls = what is here? • cd = move • mkdir = make folder • touch = make file

File operations: cp = copy • mv = move/rename • rm = remove • cat = display • less = read page-by-page

Search & information: grep = search text • find = search files • head = first lines • tail = last lines • history = previous commands • man = manual

Identity: whoami = current user • id = user/group IDs • groups = group membership

Permissions: r=4, w=2, x=1 • 7=rwx • 6=rw- • 5=r-x • 4=r-- • 0=---

Common chmod: 600 = owner read/write only • 644 = owner read/write, others read • 700 = owner full access • +x = add execute

Users & groups: owner controls one account's ownership; groups make shared access easier; least privilege = only necessary access

Python security: validate input • avoid eval/exec on untrusted input • don't hard-code secrets • use trusted libraries • don't log passwords

Password safety: long + unique + unpredictable • don't reuse • enable MFA • don't use real passwords in practice examples

Password storage: never plain text • use a unique salt + dedicated password hashing such as Argon2id/scrypt/bcrypt • SHA-256 alone is not a password-storage solution

CIA Triad: Confidentiality = private • Integrity = correct • Availability = accessible

Authentication vs Authorization: Authentication = who are you? • Authorization = what can you access?

Threats: phishing • malware • brute force • social engineering • weak passwords • misconfiguration

Golden rule: Learn and test only in systems and environments where you have permission.

Command	Meaning	Example
pwd	Current location	pwd
ls -l	Detailed files + permissions	ls -l
cd ..	Parent directory	cd ..
mkdir	Create directory	mkdir lab
touch	Create file	touch test.txt
cp	Copy	cp a.txt b.txt
mv	Move/rename	mv old.txt new.txt
rm	Remove	rm test.txt
grep	Search text	grep FAILED log.txt
chmod	Change permissions	chmod 600 notes.txt
whoami	Current user	whoami
id	User/group IDs	id
man	Command manual	man chmod